DBT PORTFOLIO PROJECT

LAYER: INTERMEDIATE

DBT Intermediate Model

Exponential Smoothing | Time Series | LAG() | Forecasting | Snowflake

Project Summary

Applies weighted exponential smoothing with alpha=0.3 to the healthcare claims metric using up to 10 historical lags. Each lag k periods ago is weighted by alpha*(1-alpha)^k, so recent observations carry more weight while older ones decay exponentially. Produces a smoothed forecast value for every entity/date.

Thomas Foster

Senior Data Analyst | Snowflake | DBT | Healthcare Analytics

github.com/thomasfoster | thomasfoster79@yahoo.com

1 PROJECT OVERVIEW

Model Name:	int_exponential_smoothing
Layer:	Intermediate
Materialization:	view
Smoothing Parameter:	alpha = 0.3 (configurable in model)
Lag Depth:	10 periods (increase for slower-decaying series)
Key Output:	es_smoothed_value_alpha_0_3 per entity/date
Audience:	Finance forecasting, Operations planning, Actuarial

2 BUSINESS PROBLEM

Claims data contains noise from late submissions, payment delays, and seasonal effects. Simple moving averages treat all periods equally -- but last month's claims are more predictive of next month than claims from 12 months ago. Exponential smoothing weights recent data more heavily, producing forecasts that adapt faster to trend changes while still damping noise.

- alpha=0.3: Moderate smoothing -- reacts to changes within ~3-4 periods. Increase for more responsiveness.
- Lag depth of 10: Captures ~10 periods of decaying history. Weights beyond lag 10 are negligible at alpha=0.3.
- Comparison to MA: MA weights all periods equally; ES weights geometrically -- more realistic for claims.
- Output feeds into the mart alongside MA columns for side-by-side comparison.

3 TECHNICAL EXPLANATION

- source_data: Reads claims source; adds ROW_NUMBER() per entity for ordering (not directly used in smoothing).
- exponential_smoothing CTE: Manually expands the ES formula across 10 lags.
- Weight for lag k = 0.3 * (0.7)^k. Lag 0 = 0.3, lag 1 = 0.21, lag 2 = 0.147 ... lag 10 = ~0.0028.
- LAG(s.metric_value, k) OVER (PARTITION BY s.entity_id ORDER BY s.period_date) retrieves each historical observation.
- POWER(0.7, k) computes the decay factor for each lag position.
- Sum of all 11 weighted terms = es_smoothed_value_alpha_0_3. NULL lags at series start are handled gracefully by Snowflake (NULL + number = NULL, not error).

Key Techniques: LAG() multi-lag expansion, POWER() decay weights, PARTITION BY entity_id, configurable alpha

4 FULL MODEL CODE

```
{{ config(materialized='view') }}

with source_data as (
    select
        entity_id, period_date, metric_value,
        row_number() over (
            partition by entity_id order by period_date
        ) as rn
    from {{ source('healthcare', 'claims') }}
),

-- Alpha = 0.3; weight for k periods ago = 0.3 * (0.7)^k
exponential_smoothing as (
    select
        s.entity_id, s.period_date, s.metric_value,
        (
            0.3                    * s.metric_value
            + 0.3 * 0.7            * lag(s.metric_value, 1) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7, 2)  * lag(s.metric_value, 2) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7, 3)  * lag(s.metric_value, 3) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7, 4)  * lag(s.metric_value, 4) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7, 5)  * lag(s.metric_value, 5) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7, 6)  * lag(s.metric_value, 6) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7, 7)  * lag(s.metric_value, 7) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7, 8)  * lag(s.metric_value, 8) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7, 9)  * lag(s.metric_value, 9) over (partition by s.entity_id order by s.period_date)
            + 0.3 * power(0.7,10)  * lag(s.metric_value,10) over (partition by s.entity_id order by s.period_date)
        ) as es_smoothed_value_alpha_0_3
    from source_data s
)

select * from exponential_smoothing order by entity_id, period_date
```